A

Minor Project Synopsis on

TrustCart: An AI-Powered Product Authenticity & Price Intelligence Platform

Submitted to

RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA, BHOPAL (M.P)

In Partial fulfillment for the award of degree

of

BACHELOR OF TECHNOLOGY

IN

DATA SCIENCE

By

Vishesh Kumar Dixit, 0818CD231070

Dhairya Borse, 0818CD243D01

Under the Guidance of

Mr. Shreyas Pagare

Professor

INDORE INSTITUTE OF SCIENCE & TECHNOLOGY, INDORE

PITHAMPUR ROAD, OPPOSITE IIM, RAU, INDORE 453331, MP.

Approved by AICTE, New Delhi, affiliated to RGPV, Bhopal, Recognized by UGC under Section 2(f)

Jul-Dec-2025

DECLARATION

We hereby declare that the work, which is being presented in this dissertation entitled **"TrustCart: An AI-Powered Product Authenticity & Price Intelligence Platform"** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in Data Science, is an authentic record of work carried out by us.

The matter embodied in this report has not been submitted by us for the award of any other degree.

Vishesh Kumar Dixit
0818CD231070

Dhairya Borse
0818CD243D01

Department of Data Science

CERTIFICATE

This is to certify that the project entitled **"TrustCart: An AI-Powered Product Authenticity & Price Intelligence Platform"** submitted to RGPV, Bhopal (M.P.) in the Department of **Data Science** by

Mr. Vishesh Kumar Dixit **Enrollment Number 0818CD231070**

Mr. Dhairya Borse **Enrollment Number 0818CD243D01**

in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology** in **Data Science** during the academic year 2025-2026.

PROJECT GUIDE **HEAD OF THE DEPARTMENT**

PRINCIPAL

ABSTRACT

In today's world reviews on online websites play a vital role in sales of the product because people try to get all the pros and cons of any product before they buy it as there are many different options for the same product as there can be different manufactures for the same type of product or there might be difference in sellers that can provide the product or there might be some difference in the procedure that is taken while buying the product so the reviews are directly related to the sales of the product and thus it necessary for the online websites to spot fake reviews as it's their own reputation that comes into consideration as well, so a Fake Review Detection is used to spot any fraudulent going on because it's not possible for them to verify every product and sale manually so a program comes into the picture that tries to detect any pattern in the reviews given by the customers.

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to everyone who has supported us throughout the journey of completing our minor project.

First and foremost, we would like to thank the Head of the Department, **Dr. Reeshu Agrawal**, for their encouragement, guidance, and providing us with all the necessary facilities to carry out this project.

We extend our sincere gratitude to our Project Coordinator, **Mrs. Manisha Kadam**, for their continuous support, timely advice, and valuable feedback. We are deeply indebted to our Project Guide Mr. Shreyas Pagare, for their expert guidance, valuable suggestions, and support throughout this project.

Thank you all for making this project a meaningful and enriching experience.

Vishesh Kumar Dixit ***0818CD231070***
Dhairya Borse ***0818CD243D01***

LIST OF TABLES

LIST OF FIGURES

I N D E X

CHAPTER **PAGE NO**

CHAPTER 1

INTRODUCTION

1.1 Background

In recent years, online platforms such as e-commerce websites, travel portals, and service review platforms have become an important part of everyday decision-making. Customers heavily rely on online reviews to evaluate product quality, service reliability, and overall user experience. Positive reviews can significantly increase product sales, while negative reviews can reduce customer interest. Due to this strong influence, online reviews play a critical role in shaping consumer behavior and business reputation.

However, the growing importance of online reviews has also led to the rapid increase of fake reviews. Fake reviews are intentionally written to mislead users by promoting or demoting products and services unfairly. These reviews may be generated by competitors, paid reviewers, or automated bots. As a result, customers may make incorrect purchasing decisions, and genuine businesses may suffer financial and reputational losses. Detecting fake reviews manually is extremely difficult because of the massive volume of user-generated content produced every day.

Traditional methods of identifying fake reviews, such as manual moderation and keyword-based filtering, are no longer effective. These approaches are time-consuming and unable to handle large datasets efficiently. Fake reviews are often written in natural language that closely resembles genuine reviews, making them difficult to detect using simple rule-based techniques. This limitation highlights the need for intelligent and automated solutions.

Machine Learning and Natural Language Processing (NLP) provide powerful techniques to overcome these challenges. Machine learning algorithms learn patterns from historical review data, while NLP techniques help process and understand textual information. By converting review text into numerical features, models can classify reviews as fake or genuine with better accuracy. Automated fake review detection systems improve transparency, enhance customer trust, and support fair decision-making on online platforms.

Therefore, the study and development of fake review detection systems using Machine Learning and NLP have become an important research area. This project aims to explore this domain by developing a data-driven solution that effectively identifies fake reviews and contributes to improving the reliability of online review systems.

1.2 Problem Statement

As the use of online products and application has been increasing at a rapid rate, the competition in the market is increasing day by day. Business officials in order to fame their products and defame other competitor products may get people from other marketplace or hire them to post and give fake judgements on the products. So, to safeguard the authenticity of the online products and opinions various steps and methods are needed to be applied.

Financially if we look at the situation, the opinions and reviews on various products may cost a lot to one company if people are actually relying on those reviews. If the reviews are original and authentic then the results are more obvious and fairer, but if we ponder over the other side then the situation reverses. More and more fake reviews will cause a huge loss for the company which has got good and better products to sale which causes a huge financial loss to the growing company.

Fake Review Detection will help us to identify whether the opinion given to the product is a true or fake review. Various methods can be used to detect it but we our focusing on finding the results using Machine learning Techniques.

Our method focuses on the text content of the costumer reviews. People who tend to write fake reviews choose topics or words to influence online customers thus their word choice is different than others. This word choice pattern can be used for segregation of fake and truthful reviews. If these reviews are detected properly, then fake reviews may be automatically removed after detection, which may help to generate only the genuine information and serve as a boon to the companies and market especially the customers.

1.3 Purpose / Objective

The main purpose of this project is to design and develop an automated system that can detect fake reviews on online platforms using Machine Learning and Natural Language Processing techniques. With the rapid growth of e-commerce and digital services, online reviews have become a major factor influencing customer decisions. This project aims to reduce the influence of misleading and fraudulent reviews by providing a reliable method for identifying genuine and fake review content.

The project focuses on analyzing review text to understand common patterns and characteristics associated with fake reviews. Natural Language Processing techniques are used to preprocess and extract meaningful

information from textual data, while machine learning algorithms are applied to classify reviews accurately. Through this approach, the system aims to improve the overall quality and credibility of online review platforms, enhance user trust, and support fair decision-making for customers and businesses.

1.4 Solution Approach

The proposed solution follows a data-driven approach to detect fake reviews using Machine Learning and Natural Language Processing techniques. The system begins with the collection of a labeled dataset containing both fake and genuine reviews from reliable online sources. This dataset forms the foundation for training and evaluating the machine learning models.

Once the data is collected, preprocessing is performed to clean and prepare the review text for analysis. This includes removing unnecessary symbols, stop words, and irrelevant characters, as well as converting the text into a standardized format. Natural Language Processing techniques such as tokenization and feature extraction are applied to transform textual data into numerical representations that can be understood by machine learning algorithms.

After preprocessing, machine learning models are trained using the extracted features to learn patterns that differentiate fake reviews from genuine ones. Algorithms such as Logistic Regression, Naive Bayes, or Support Vector Machines are used for classification. The trained model is then tested on unseen data to evaluate its performance using accuracy and other evaluation metrics. Based on the predictions, the system classifies each review as fake or genuine, providing an automated and efficient solution for fake review detection.

1.4.1Scope of the Project Includes:

- Collection and use of publicly available review datasets for analysis.
- Preprocessing of review text using Natural Language Processing techniques.
- Extraction of meaningful features from textual data for classification.
- Training and testing of machine learning models to detect fake reviews.
- Classification of reviews as fake or genuine based on learned patterns.
- Performance evaluation of the model using standard accuracy metrics.

1.5 Existing System & Limitations:

The existing systems used for handling online reviews mainly rely on manual moderation, user reporting, and basic rule-based filtering techniques. In many platforms, reviews are checked manually by administrators or flagged by users if they appear suspicious. Some systems use simple keyword matching or rating-based rules to identify unusual reviews. While these methods help to a limited extent, they are not effective for large-scale platforms where thousands of reviews are generated every day.

The major limitation of the existing system is its inability to accurately detect fake reviews written in natural and convincing language. Manual review processes are time-consuming, prone to human error, and not scalable. Rule-based systems fail when fake reviews do not contain obvious keywords or follow predefined patterns. Additionally, traditional systems do not adapt to new writing styles or evolving fake review strategies. As a result, many fake reviews remain undetected, leading to reduced user trust and unreliable decision-making on online platforms.

1.5.1 Existing System Characteristics:

- **Manual Review Moderation**
  The existing system largely depends on manual checking of reviews by administrators or moderators. This process requires human effort and is time-consuming. It becomes inefficient when handling a large number of reviews.
- **Rule-Based Filtering Techniques**
  Many platforms use simple rule-based methods such as keyword matching or rating thresholds to identify suspicious reviews. These rules are fixed and cannot adapt to new patterns. As a result, fake reviews written in natural language often go undetected.
- **Limited Scalability**
  Traditional review management systems are not designed to handle massive volumes of data efficiently. As the number of users increases, system performance degrades. This limitation makes existing systems unsuitable for large-scale online platforms.

1.5.2 Limitations of the Existing System:

- **High Dependence on Manual Effort**
  Existing systems rely heavily on human moderators to review and verify online reviews. This

process is slow and requires significant manpower. It also increases the chances of human error and inconsistency.

- **Low Accuracy in Detecting Fake Reviews**
  Rule-based and manual systems struggle to accurately identify fake reviews written in natural language. Such systems fail when fake reviews appear genuine. This leads to many fraudulent reviews remaining undetected.
- **Lack of Scalability**
  Traditional systems are not capable of efficiently handling large volumes of review data. As the number of users and reviews increases, system performance degrades. This makes them unsuitable for large online platforms.
- **Inability to Adapt to New Patterns**
  Fake review strategies continuously evolve over time. Existing systems use fixed rules and cannot learn from new data. As a result, they become outdated quickly.
- **Time-Consuming Review Process**
  Manual and semi-automated review methods require significant processing time. Delayed detection allows fake reviews to influence customer decisions. This reduces trust in the platform.

CHAPTER 2

METHODOLOGY

2.1 Proposed Methodology

The method applied focus on the words of the review which different users use to write their reviews. Both the review is written for different purposes therefore their choice of words is also different. In order to create impact on others fake review, writers write review with a lot of information (hence their reviews are large in size). Also, the chose words that will attract most of the customers to read them before buying a product. Words will include adjectives, adverbs and also words for comparison on the other hand a genuine review writer will write reviews based on its personal experience thus include more nouns, verbs and less of comparisons. This choice of words can be used to distinguish between these two kinds of reviews and help in review spam detection. The dataset contains large number of reviews from seven different business domains. Hundred topics are extracted from theses reviews corresponding to each domain by using Latent Dirichlet Allocation. These hundred topics are then used as train models to classify individual review by calculating the score of each individual review. We have two sub collection of reviews true type (DT and fake type DF) Based on the formula: -

$$\Theta_X = \frac{\left(\sum_{d \in D_X} \theta_d\right)}{s}$$

θ is topic distribution of test review

θt is the global topic distribution of true review w.r.t LDA topics.

θf is the global topic distribution of true review w.r.t LDA topics
X={T,F} and / is element wise division.

The factors $wT\ i$ and $wF\ i$ are weights of the i - th topics in $\boldsymbol{DT}$ and $\boldsymbol{DF}$, respectively. The weights reflect how widely a topic is spread across the reviews in either sub collection (True and fake)

$wXi=\log(|\{d \in \boldsymbol{D}X : \theta d\ i > \tau\}|+1/|\{d \in \boldsymbol{D}Y : \theta d\ i > \tau\}|+1)$

where $\{X,Y\} = \{T,F\}$ or$\{F,T\}$ and τ=1/K K is the number of topics. Finally, we combine the two factors for the final score:

$$\text{ScoreX}(r) = \Sigma\theta r\, i \times (\Theta X\, i + \sigma \times wX\, i) \text{ where } X = \{T, F\}$$

θr is topic distribution of test review r, which is the result of the inference in LDA. We consider that the global trend of topics would be more important and therefore wTi and $\diamond\diamond Fi$ is multiplied by σ (0.2) in the current implementation. The decision of whether or not r belongs to the fake review category is determined by comparing its scores for fake and truthful reviews: $scoreF\ r > scoreT\ r$

Apart from calculating the score the word choice pattern can also be used to distinguish between the reviews. All the reviews mails use five kinds of word type. These five categories are: -

- Concrete Experience (CE): Topics in this class mainly contain verbs.
- Detailed Information (DI): Topics usually composed of specific nouns.
- General Comments (GC): Topics contains describing abstract evaluations and adjectives.
- Comparative Assessment (CA): Topics in this class include comparative words like better, easier etc.
- Reference and Recommendation (RR): Includes words like refer, advice etc.

2.2 Process Model Adopted:

To develop an effective fake review detection system, it is essential to follow a structured approach that guides the project from understanding the problem to deploying the solution. For this purpose, the CRISP-DM (Cross-Industry Standard Process for Data Mining) model is adopted, as it provides a systematic framework for data-driven projects like ours.

1. Business Understanding

- **Goal:** Detect fake reviews to improve trust in online platforms.
- **Objectives:** Identify misleading or fraudulent reviews using ML and NLP techniques.
- **Constraints:** Limited labelled data, noisy text, varying review formats.

2. Data Understanding

- **Data sources:** Amazon/Yelp/Kaggle datasets containing reviews, ratings, timestamps, and user IDs.
- **Data exploration:** Check for missing values, analyse review length, sentiment, and user activity.

- **Insight:** Identify patterns of fake reviews (e.g., overly positive/negative, repeated words, unusual posting times).

3. Data Preparation

- **Tasks:**
    - Clean text: remove punctuation, stop words, emojis, and special characters.
    - Preprocess: tokenization, lemmatization/stemming, lowercasing.
    - Feature engineering: TF-IDF, word embeddings, sentiment scores, review metadata features.
    - Split dataset into training and testing sets.

4. Modelling

- **Algorithms:**
    - Classical ML: Logistic Regression, Random Forest, SVM.
    - Advanced NLP: LSTM, BERT, or transformer-based classifiers.
- **Iteration:** Tune hyperparameters, test multiple models to find the best-performing one.

5. Evaluation

- **Metrics:** Accuracy, Precision, Recall, F1-score, ROC-AUC.
- **Validation:** Use test data to ensure model generalization.
- **Goal:** Ensure the model reliably identifies fake reviews before deployment.

6. Deployment

- **Implementation:** Deploy as a web application using Flask/Django or as an API.
- **User interaction:** Users can input reviews to check if they are fake or genuine.
- **Monitoring:** Track model performance and update periodically with new data.

7. Iteration & Maintenance

- Continuously collect new reviews, retrain and fine-tune the model.
- Adjust features and models based on performance and new types of fake reviews.

2.3 Planning and Scheduling:

The project is divided into 12 weeks to ensure systematic progress, proper testing, and timely completion. Each week focuses on specific tasks to gradually build the fake review detection system.

Weeks 1-2 – Requirement Analysis & Project Planning

- Understand the problem of fake reviews and define project objectives.
- Identify scope, constraints, and success metrics.
- Plan resources, tools, and dataset requirements.

Weeks 3-4 – Data Collection

- Collect review datasets from sources like Amazon, Yelp, or Kaggle.
- Ensure variety in reviews (different products, users, time periods).
- Perform initial exploration to identify missing data, anomalies, or patterns.

Weeks 5-6 – Data Preprocessing

- Clean and preprocess text: remove punctuation, stopwords, emojis, and special characters.
- Perform NLP preprocessing: tokenization, lemmatization, and lowercasing.
- Handle missing data and normalize features for consistency.

Weeks 7-8 – Feature Engineering & Selection

- Extract relevant features: review length, sentiment scores, user activity patterns.
- Create NLP features: TF-IDF vectors, word embeddings (Word2Vec, GloVe).
- Apply feature selection techniques to reduce dimensionality and improve model efficiency.

Weeks 9-10 – Model Development & Training

- Implement classical ML models: Logistic Regression, Random Forest, SVM.
- Train advanced NLP models: LSTM or BERT for text classification.
- Tune hyperparameters and evaluate models using cross-validation.

Week 11 – Evaluation & Testing

- Evaluate models using metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC.
- Compare results of different models and finalize the best-performing one.
- Conduct testing with unseen reviews to validate performance.

Week 12 – Deployment & Maintenance Planning

- Deploy the model using Flask/Django as a web application or API.
- Enable user input for review checking and prediction.
- Plan maintenance: periodic retraining with new data and updating features to handle evolving fake reviews.

CHAPTER 3

REQUIRENMENT AND ANALYSIS

3.1 Problem Definition:

With the rapid growth of e-commerce and online review platforms, customers heavily rely on user reviews to make purchasing decisions. However, the increasing prevalence of fake or misleading reviews poses a serious challenge, as they can manipulate perceptions, misguide consumers, and affect the reputation of products and sellers.

Detecting these fraudulent reviews manually is time-consuming, error-prone, and impractical due to the massive volume of online reviews. Therefore, there is a need for an automated system that can accurately identify fake reviews using advanced techniques in Natural Language Processing (NLP) and Machine Learning (ML).

The main problem addressed in this project is to design and develop a reliable model that can differentiate between genuine and fake reviews, ensuring that customers receive trustworthy information and helping businesses maintain credibility.

Existing Problem	Description	Proposed Solution in Our Project
Fake or misleading reviews	Reviews written to intentionally mislead customers, either to promote or demote a product	Use ML and NLP-based models to automatically classify reviews as genuine or fake
Manual review detection	Human moderation is time-consuming and cannot handle large volumes of reviews	Automated system processes thousands of reviews quickly with high accuracy
Unstructured review text	Reviews often contain informal language, slang, emojis, and grammatical errors	Preprocessing techniques like tokenization, lemmatization, and text cleaning normalize the data for analysis

Existing Problem	Description	Proposed Solution in Our Project
Difficulty in detecting patterns	Fake reviews often follow subtle patterns that are hard to detect manually	Feature engineering (review length, sentiment, user activity patterns) helps models learn distinguishing characteristics
Lack of scalability	Platforms cannot efficiently monitor and verify all incoming reviews	Machine learning models can be deployed on web applications to scale detection in real-time
Constantly evolving fake reviews	Fraudsters change writing styles over time to bypass detection	Iterative model updates and retraining with new data keep the system adaptive

3.2 Feasibility Study:

A feasibility study evaluates whether the proposed fake review detection system can be successfully developed and implemented within the given constraints. This study covers technical, operational, and economic feasibility.

3.2.1 Technical Feasibility:

Technical feasibility assesses whether the system can be implemented using existing technology, tools, and resources. For the Fake Review Detection project, the technical feasibility is high due to the availability of mature frameworks, libraries, and computational resources. The system primarily relies on Python; a versatile programming language widely used for machine learning and natural language processing (NLP) tasks. Python supports multiple NLP libraries, such as NLTK, spaCy, and TextBlob, which facilitate text preprocessing, tokenization, lemmatization, and sentiment analysis. These preprocessing steps are essential for cleaning and structuring the review data before feeding it into machine learning models.

For the machine learning component, libraries such as Scikit-learn provide tools to implement classical models like Logistic Regression, SVM, and Random Forest, while PyTorch or TensorFlow enable training advanced deep learning models like LSTM or BERT for text classification. The availability of pre-trained embeddings and transformers significantly reduces model development time and enhances accuracy.

The project also considers data availability, which is a critical factor in technical feasibility. Public datasets from platforms such as Amazon, Yelp, and Kaggle offer labeled review data that can be used to train and validate models. Additionally, web scraping techniques can be employed to collect more up-to-date reviews for continual model improvement.

For deployment, the system can be implemented as a web application using frameworks like Flask or Django, which allow the trained model to be integrated into an interactive interface. Users can input review text and instantly receive predictions on whether a review is genuine or fake. The system can also be scaled to handle multiple requests using cloud-based platforms if needed.

Finally, computational requirements for this project are reasonable. Training classical ML models can be performed on standard personal computers, while deep learning models may require GPUs or cloud services for faster processing. Overall, the combination of open-source tools, accessible datasets, and moderate hardware requirements makes the project highly technically feasible.

Aspect	**Details**	**Relevance to Project**
Programming Language	Python	Widely used for ML and NLP; supports all required libraries
NLP Libraries	NLTK, spaCy, TextBlob	For text cleaning, tokenization, lemmatization, and sentiment analysis
Machine Learning Libraries	Scikit-learn, PyTorch, TensorFlow	Enables classical ML and deep learning models like Logistic Regression, SVM, Random Forest, LSTM, BERT
Data Availability	Amazon, Yelp, Kaggle datasets	Provides labeled review data for training and testing

Aspect	Details	Relevance to Project
Deployment Tools	Flask, Django	Integrates model into a user-friendly web application
Hardware Requirements	Standard PC or GPU-enabled system	Classical models run on PCs; deep learning models may need GPU/cloud for efficiency
Scalability	Cloud deployment possible	System can handle large volumes of reviews for real-time detection

3.2.2 Economic Feasibility

Economic feasibility evaluates the financial aspects of developing and implementing the project. It helps determine whether the project is cost-effective and provides value compared to its cost. For the Fake Review Detection project, the economic feasibility can be analyzed under the following headings:

1. Software Cost

- The project primarily uses open-source tools and libraries such as Python, NLTK, spaCy, Scikit-learn, PyTorch, and Flask/Django.
- No paid software is required, which reduces software licensing costs to zero.

2. Hardware Cost

- Classical machine learning models can run on standard personal computers, which are already available.
- Deep learning models like LSTM or BERT may require GPU resources, which can be accessed via cloud platforms like Google Colab or AWS on a pay-per-use basis.
- Overall hardware cost is minimal compared to project benefits.

3. Data Acquisition Cost

- Publicly available datasets from Amazon, Yelp, and Kaggle are used for training and testing.

- No paid data acquisition is necessary, making the data component cost-efficient.

4. Development and Maintenance Cost

- Since the team is handling development in-house, there is no additional labor cost.
- Maintenance mainly involves updating the model with new reviews periodically, which can be done with minimal effort.

5. Benefit Analysis

- Automating fake review detection saves manual effort and ensures faster, more reliable results.
- Improves trustworthiness of reviews, benefiting both consumers and e-commerce platforms.
- The minimal investment in software and hardware is outweighed by the value provided by accurate and scalable review detection.

3.2.3 Operational Feasibility

Operational feasibility evaluates whether the proposed system can be effectively integrated into the existing environment and whether it meets user requirements in a practical way. For the Fake Review Detection project, operational feasibility can be analyzed under the following points:

1. User Requirements

- The system is designed to automatically detect fake reviews, fulfilling the need of e-commerce platforms, review aggregators, and consumers for trustworthy reviews.
- Users can input reviews and instantly receive a prediction, making the system user-friendly and practical.

2. Ease of Use

- The system can be deployed as a web application or API, allowing users to interact with it without requiring technical expertise.
- Clear input and output interfaces ensure smooth operation and minimal user training.

3. Reliability and Accuracy

- Machine learning and NLP models provide consistent and accurate predictions, reducing errors compared to manual detection.
- The system can handle large volumes of reviews, ensuring operational efficiency for platforms with high traffic.

4. Integration with Existing Systems

- The application can be easily integrated with e-commerce platforms or review management systems to provide real-time fake review detection.
- Modular design allows for upgrades or additions such as incorporating new NLP models or features in the future.

5. Maintenance and Support

- Periodic retraining of the model with new review data ensures the system remains up-to-date with evolving fake review patterns.
- Minimal operational support is required since the system is largely automated.

3.3 Functional Requirements

Functional requirements describe the specific functionalities and capabilities that the **Fake Review Detection system** must provide to meet its objectives. These requirements ensure that the system performs the tasks needed to detect fake reviews accurately and efficiently.

1. User Input Handling

- The system must allow users to input reviews manually or upload datasets of reviews.
- It should support various formats, including text files, CSV, or direct text entry on a web interface.

2. Text Preprocessing

- The system must clean and preprocess the review text to remove noise such as punctuation, special characters, stopwords, and emojis.
- It should tokenize, lemmatize/stem, and normalize text to prepare it for machine learning models.

3. Feature Extraction

- The system must extract relevant features from reviews, including review length, sentiment score, frequency of repeated words, and user activity patterns.
- NLP-based features like TF-IDF vectors, Word2Vec embeddings, or transformer-based representations must be generated for model training.

4. Fake Review Classification

- The system must classify reviews as **genuine or fake** using trained machine learning or deep learning models.
- The classification process must be accurate, with predictions based on both textual content and metadata features.

5. Model Training and Evaluation

- The system must support model training on labeled datasets and evaluate performance using metrics such as **Accuracy, Precision, Recall, F1-score, and ROC-AUC**.
- It should allow comparison of multiple models to select the best-performing one.

6. Result Display and Reporting

- The system must display classification results to the user in a clear and understandable format.
- It should generate reports summarizing model performance, number of fake reviews detected, and other key metrics.

7. Deployment and User Interaction

- The system must be deployable as a **web application or API**, enabling real-time detection for individual or bulk reviews.

- Users should receive instant predictions, ensuring operational efficiency.

8. Scalability and Maintenance

- The system must allow easy retraining of models with new data to handle evolving fake review patterns.
- It should support handling large volumes of reviews without performance degradation.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance standards, and operational constraints of the **Fake Review Detection system**. They ensure that the system is efficient, reliable, and user-friendly in addition to performing its core functionalities.

1. Performance

- The system must provide real-time or near real-time predictions for single or batch review inputs.
- It should process and classify thousands of reviews efficiently without significant delays.

2. Reliability

- The system should maintain consistent accuracy in detecting fake reviews across various datasets.
- It must handle errors gracefully, such as missing data or unsupported input formats, without crashing.

3. Scalability

- The system should be able to scale for increasing volumes of reviews, ensuring performance does not degrade when handling large datasets.
- It must allow for easy integration of improved models or additional features in the future.

4. Usability

- The system must have a **user-friendly interface** that is easy to navigate for both technical and non-technical users.

- It should provide clear instructions for inputting data and interpreting predictions.

5. Security

- The system must ensure that user data, including reviews and uploaded datasets, is handled securely and not exposed to unauthorized access.
- It should comply with standard data privacy practices.

6. Maintainability

- The system should allow for easy updates and retraining of the models with new data.
- Code should be modular and well-documented to simplify maintenance and debugging.

7. Availability

- The deployed system should be accessible via web or API **24/7** for continuous use.
- It should handle multiple user requests simultaneously without downtime.
- **8. Portability**
- The system should be deployable on different platforms, including local servers and cloud environments, without requiring significant modifications.

3.5 Technical Requirements

Technical requirements specify the hardware, software, and tools needed to successfully develop, implement, and deploy the **Fake Review Detection system**. These requirements ensure that the project can be executed efficiently and meet the desired performance standards.

1. Hardware Requirements

- **Processor:** Minimum Intel i5 or equivalent for model training and data preprocessing.
- **RAM:** At least 8 GB for handling large datasets; 16 GB recommended for deep learning models.
- **Storage:** Minimum 256 GB for storing datasets, models, and project files; additional storage may be required for large datasets.

- **GPU (Optional):** NVIDIA GPU with CUDA support for faster training of deep learning models (e.g., LSTM or BERT).

2. Software Requirements

- **Operating System:** Windows 10 or above / Linux / macOS
- **Programming Language:** Python 3.x
- **Libraries for NLP:** NLTK, spaCy, TextBlob
- **Libraries for Machine Learning:** Scikit-learn, PyTorch, TensorFlow
- **Web Frameworks:** Flask or Django for deployment
- **Data Handling:** Pandas, NumPy for data manipulation
- **Visualization:** Matplotlib, Seaborn, or Plotly for analysis and reporting

3. Development Tools

- **IDE:** Visual Studio Code, PyCharm, or Jupyter Notebook for coding and experimentation.
- **Version Control:** Git/GitHub for managing project code and collaboration.
- **Package Management:** pip or conda for installing required libraries and dependencies.

4. Data Requirements

- Publicly available datasets from **Amazon, Yelp, or Kaggle**.
- Review data should include text, rating, user ID, product ID, and other metadata.
- Data must be **cleaned and preprocessed** before model training.

5. Deployment Requirements

- **Web Server:** Local server or cloud-based deployment (Heroku, AWS, or Google Cloud).
- **Browser Compatibility:** Modern web browsers (Chrome, Firefox, Edge) for accessing the web interface.
- **API Support:** REST API endpoints for integration with other applications if required.

6. Security Requirements

- Secure handling of user data and uploaded datasets.
- Protection against unauthorized access and ensuring data privacy.

3.5.1 Software Requirements:

The **software requirements** define the tools, libraries, frameworks, and platforms necessary to develop, train, test, and deploy the **Fake Review Detection system**. These requirements ensure smooth execution of all project tasks and compatibility with the chosen technologies.

1. Programming Language

- **Python 3.x** – Chosen for its extensive support in **machine learning (ML)** and **natural language processing (NLP)**. Python provides libraries for text preprocessing, model training, evaluation, and deployment.

2. NLP Libraries

- **NLTK (Natural Language Toolkit):** Tokenization, stopwords removal, stemming, and text preprocessing.
- **spaCy:** Efficient NLP processing including lemmatization, named entity recognition, and vectorization.
- **TextBlob:** Sentiment analysis and basic text processing.

3. Machine Learning & Deep Learning Libraries

- **Scikit-learn:** For implementing classical ML models like Logistic Regression, Random Forest, and SVM.
- **PyTorch / TensorFlow:** For training advanced models such as LSTM, BERT, and other deep learning classifiers.
- **Transformers library (Hugging Face):** For pre-trained NLP models to improve fake review detection accuracy.

4. Data Handling & Analysis Libraries

- **Pandas:** Data manipulation, cleaning, and storage.
- **NumPy:** Efficient numerical computations for feature extraction and model inputs.

5. Visualization Libraries

- **Matplotlib / Seaborn / Plotly:** To create charts, graphs, and visual reports of model performance and review analytics.

6. Web Frameworks

- **Flask / Django:** For deploying the trained model as a web application or API.
- Enables real-time review input and prediction display in a user-friendly interface.

7. Development Environment & Tools

- **IDE:** Visual Studio Code, PyCharm, or Jupyter Notebook for coding and experimentation.
- **Version Control:** Git and GitHub for code management, collaboration, and versioning.
- **Package Management:** pip or conda for installing and managing project dependencies.

8. Database / Storage (Optional)

- **SQLite / MySQL / MongoDB:** For storing review data, model outputs, and user interactions if required.

3.5.2 Network & Communication Requirements:

Network and communication requirements define the infrastructure and connectivity needed to deploy and operate the **Fake Review Detection system** efficiently, especially when it is accessed as a web application or API. These requirements ensure smooth communication between users, servers, and databases, enabling real-time detection and reliable performance.

1. Internet Connectivity

- A **stable internet connection** is required for accessing cloud services, downloading datasets, and deploying the web application.
- Minimum bandwidth should be sufficient to handle multiple simultaneous requests when the system is deployed online.

2. Server Requirements

- **Web Server:** The system can be hosted on a local server or cloud-based platforms such as **AWS, Google Cloud, or Heroku**.
- **Server Specifications:** Should support Python-based frameworks (Flask/Django) and handle concurrent requests efficiently.

3. API Communication

- The system should support **RESTful APIs** for communication between the frontend interface and backend model.
- APIs will handle review input, send it to the model for classification, and return the prediction results in real-time.

4. Data Transfer and Security

- Secure protocols (HTTPS) must be used for transmitting user data to prevent interception or misuse.
- Data encryption ensures privacy and compliance with standard data protection practices.

5. Local Network (Optional)

- For offline deployment, the system can run on a **local area network (LAN)**, allowing multiple devices to access the application within the same network.
- No internet is required in this mode, but updates and new dataset integration will need periodic online access.

6. Real-time Access & Multi-user Support

- The network infrastructure should support **simultaneous multi-user access**, enabling multiple users to submit reviews and receive predictions concurrently.
- Load balancing or scalable cloud deployment can be used if user traffic is high.

CHAPTER 4

TECHNOLOGY STACK

4.1 Technology Stack Selection:

The technologies chosen for TrustCart prioritize high performance, rapid iteration, and modern design principles. The selected technologies are detailed in the table below:

Technology	Layer	Selection Rationale
React 18 & TS	Frontend	Modular component architecture, type safety, and fast rendering using a virtual DOM.
Tailwind CSS v3	Styling	Simple utility classes for responsive layouts, consistent spacing, and custom light-theme colors.
Node.js & Express	Backend	High-performance, event-driven, non-blocking I/O runtime that handles asynchronous API requests efficiently.
Google Gemini Flash	AI Engine	Fast execution times and reliable JSON schema output for data logs.
Playwright	Crawling	Controls headless Chromium browsers to load dynamic, JavaScript-heavy product pages and extract DOM elements.
PostgreSQL & Prisma	Database & ORM	Reliable relational database with transaction management. Prisma provides type-safe queries and simple migrations.
Redis & BullMQ	Task Queue	Manages background jobs reliably, preventing main thread blockages during scraping operations.

4.2 Tools and IDE

The development of the Fake Review Detection system requires several tools and Integrated Development Environments (IDEs) to facilitate coding, testing, debugging, version control, and collaboration. **Visual**

Studio Code and **PyCharm** are commonly used for writing Python code, managing libraries, and running experiments. **Jupyter Notebook** is particularly useful for **data exploration, preprocessing, and model training**, allowing step-by-step execution and visualization. **Git and GitHub** are used for version control and collaborative development, ensuring that changes are tracked and multiple team members can work simultaneously. Additionally, **package managers like pip or conda** are required for installing and managing all dependencies and libraries efficiently.

Table: Tools and IDE

Tool / IDE	Purpose	Description
Visual Studio Code	IDE	Lightweight code editor for Python development and project management
PyCharm	IDE	Professional Python IDE with debugging and testing features
Jupyter Notebook	IDE / Tool	Interactive environment for data preprocessing, exploration, and model training
Git	Version Control	Tracks changes in the code and allows collaborative development
GitHub	Repository	Hosts project code online for versioning, collaboration, and backup

4.3 Data Collection & Ingestion Pipeline

The data collection engine gathers product details from e-commerce sites. Because modern shopping websites load content dynamically using client-side JavaScript, simple HTTP requests (like axios or fetch) often fail to capture the full-page data. TrustCart solves this by using Playwright to launch headless Chromium instances that load pages, execute scripts, and wait for elements to render.

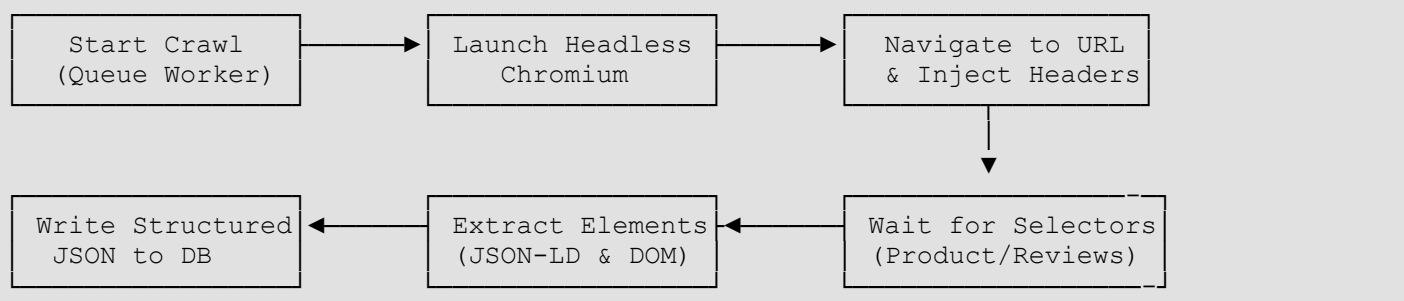

Fig: 4.1 Crawler Ingestion Sequence

The ingestion pipeline follows these steps:

1. **Request Initialization**: The worker process launches a Playwright Chromium browser. It configures user-agent strings, window sizes, and headers to mimic a standard desktop browser, reducing the risk of rate-limiting blocks.
2. **Page Navigation and Wait States**: The browser navigates to the product URL. It monitors network requests and waits for key DOM selectors (such as the product title, image container, and review list) to load.
3. **Structured Data Extraction**:
    - **JSON-LD Schema**: The scraper extracts embedded schema scripts (e.g., type application/ld+json) to parse structured details like the brand, SKU, and aggregated rating.
    - **DOM Selectors**: If the schema is missing, fallback CSS selectors extract the product title, current price, and main image.
    - **Review Extraction**: The scraper reads the reviews list, extracting the reviewer's name, date, rating, verification status, review title, and review body.
4. **Fallback Mechanism**: If a live scrape fails due to anti-bot blocks, the system loads a structured fallback mock based on the target domain, allowing the analysis pipeline to proceed.

CHAPTER 5

DIAGRAM

5.1 DATA FLOW ANALYSIS (DFD)

The Level 1 Data Flow Diagram illustrates how information moves through the TrustCart system:

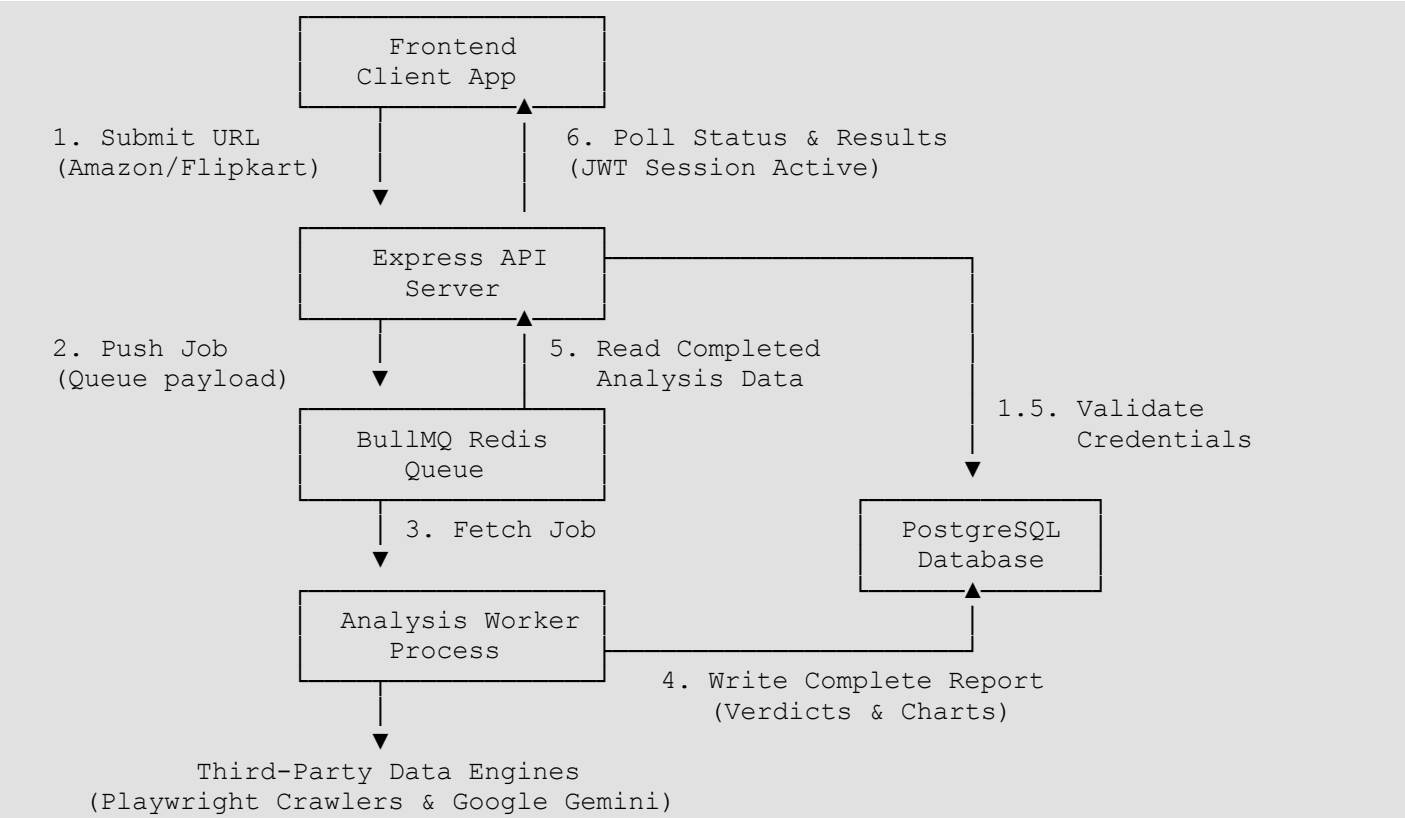

Fig: 5.1. Level 1 Data Flow Diagram (DFD)

The data flow sequence proceeds as follows:

- **Request Submission**: The user enters an e-commerce product link in the dashboard. The frontend validates the URL syntax and sends an HTTP POST request to the API server.
- **Job Registration**: The backend validates the user's JWT, creates a pending analysis record in PostgreSQL, and pushes the analysis job (containing the URL and analysis ID) to the BullMQ Redis queue.
- **Queue Management**: Redis acts as the message broker. The BullMQ worker process pulls the job from the queue to run the analysis asynchronously.
- **Task Processing**: The worker scrapes the product page, runs the review heuristic checks, queries other platforms for price comparisons, and calls the Gemini API to generate the final verdict.

- **Data Persistence**: The worker updates the PostgreSQL record status to `COMPLETED`, saves the compiled report details, and clears the Redis progress cache.
- **Results Rendering**: The frontend polls the status endpoint. Once complete, it retrieves the full report and updates the dashboard view.

5.2 DATABASE DESIGN & ENTITY RELATIONSHIP DIAGRAM (ERD)

The database design uses a normalized relational model in PostgreSQL to support quick query lookups and maintain data integrity:

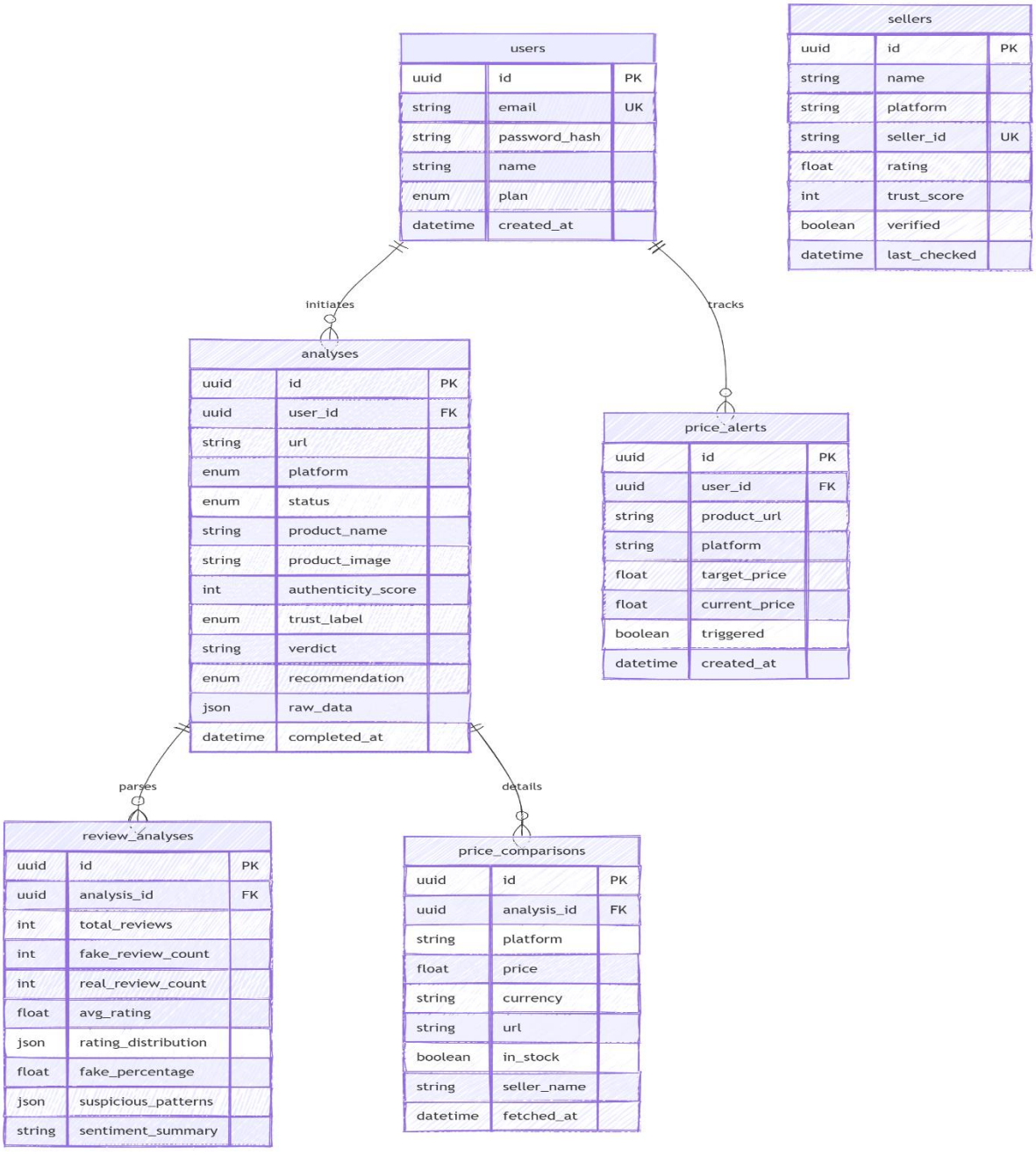

Fig: 5.2. Database Entity-Relationship Diagram (ERD)

3.3. USE CASE MODELING

The Use Case Diagram displays the interactions between users, external services, and TrustCart's core components:

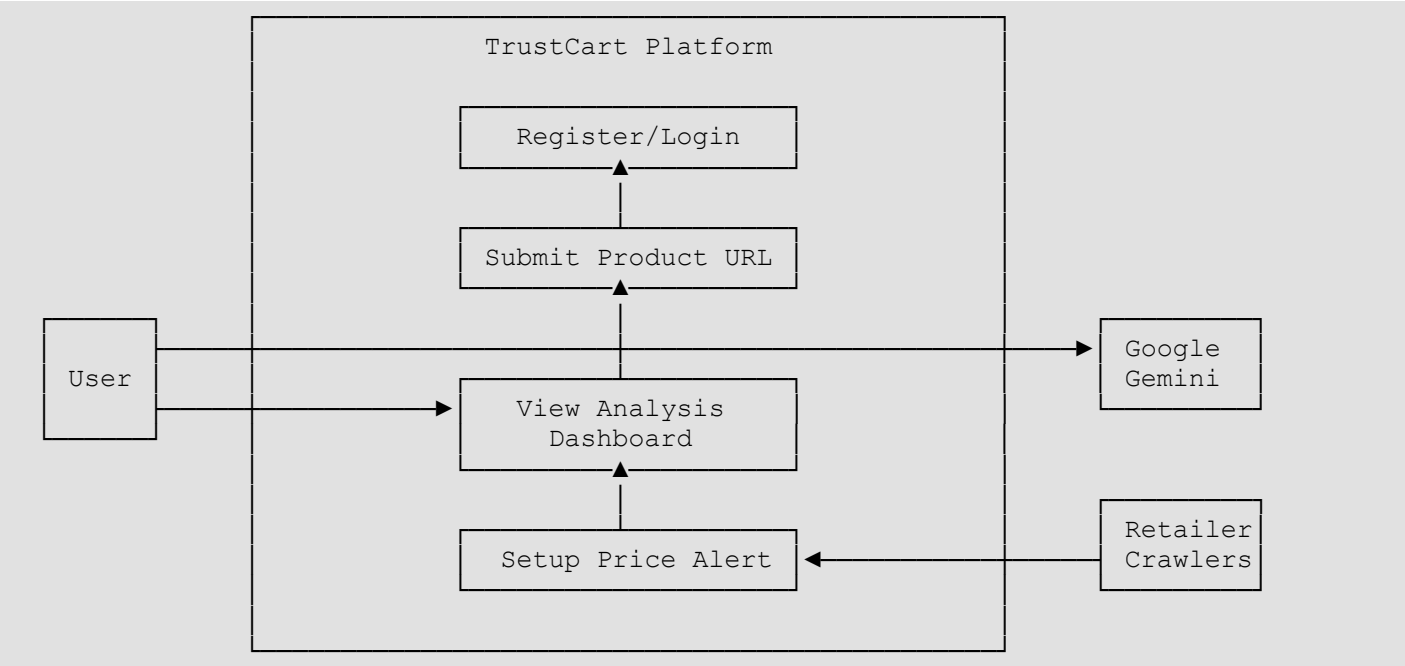

Fig: 5.3. Use Case Diagram

The system actors and use cases are detailed in the tables below:

Table: 5.2.

Actor Name	Type	Description / System Access
User (Shopper)	Human	Submit product URLs, view authenticity reports, track analyses, and configure price alerts.
Retailer Crawlers	System	Crawl e-commerce websites (via Playwright) to retrieve product and review details.
Google Gemini	System	Processes raw product details to generate structured recommendations.

Table: 5.3. Use Case Descriptions

Use Case ID	Name	Actor	Description
UC-01	Register/Login	User	Create a user account or authenticate using an email and password to receive a JWT session token.
UC-02	Submit Product URL	User	Paste an e-commerce link from a supported platform to start a background audit.
UC-03	View Analysis Dashboard	User	View the analysis results, including the authenticity score, flagged reviews, and price comparisons.
UC-04	Setup Price Alert	User	Configure target price thresholds to receive notifications when a product's price drops.

5.4. SYSTEM ARCHITECTURE

TrustCart uses a decoupled, three-tier service architecture to isolate the API server from resource-heavy web crawling and AI processing:

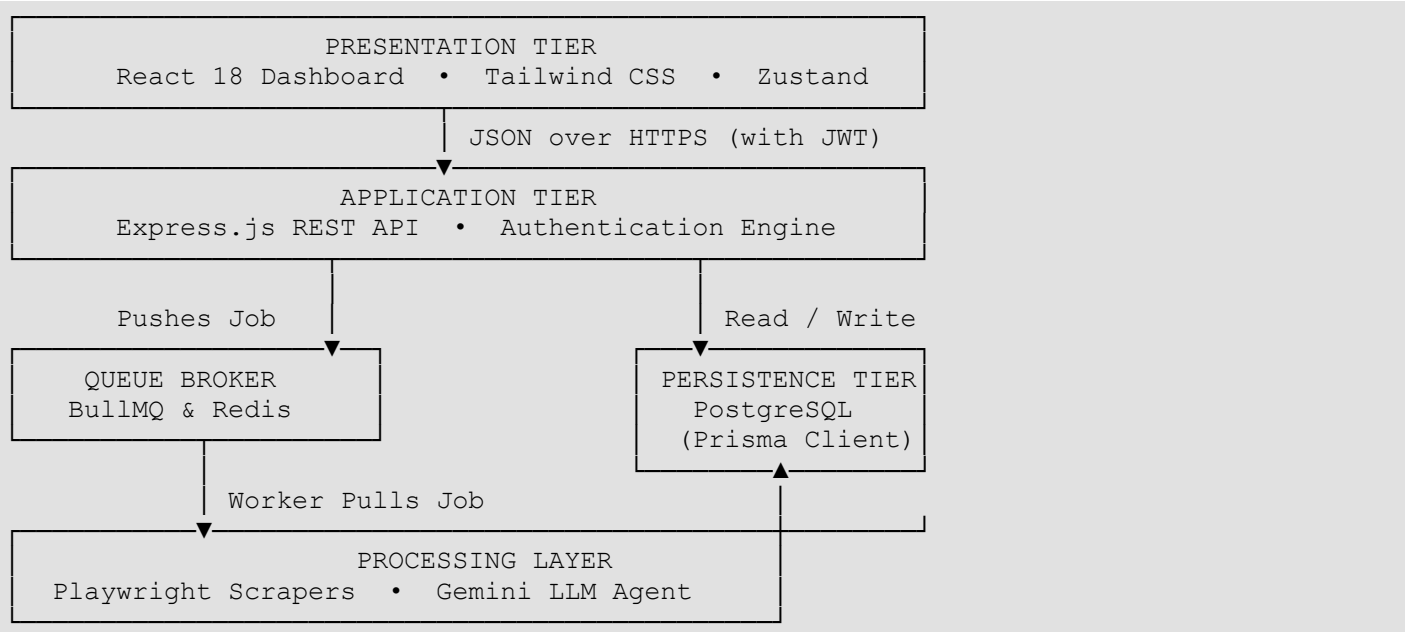

Fig: 5.4. TrustCart Layered System Architecture

The system layers function as follows:

- **Presentation Tier**: Built with **React 18, TypeScript, and Tailwind CSS** to provide a responsive user interface and communicate with the backend via REST APIs.
- **Application Tier**: An **Express.js** server that handles authentication, validates URLs, manages API requests, and schedules background analysis tasks.
- **Queue Broker**: A **Redis-powered BullMQ** queue that processes background jobs efficiently and prevents server overload.
- **Processing Layer**: Background workers scrape product data, analyze reviews, compare prices.
- **Persistence Tier**: A PostgreSQL database that stores user credentials and completed analysis reports.

5.5. PROCESS MODELING & ACTIVITY DIAGRAM

The Activity Diagram maps out the step-by-step logic followed by the background analysis worker when processing a queued job:

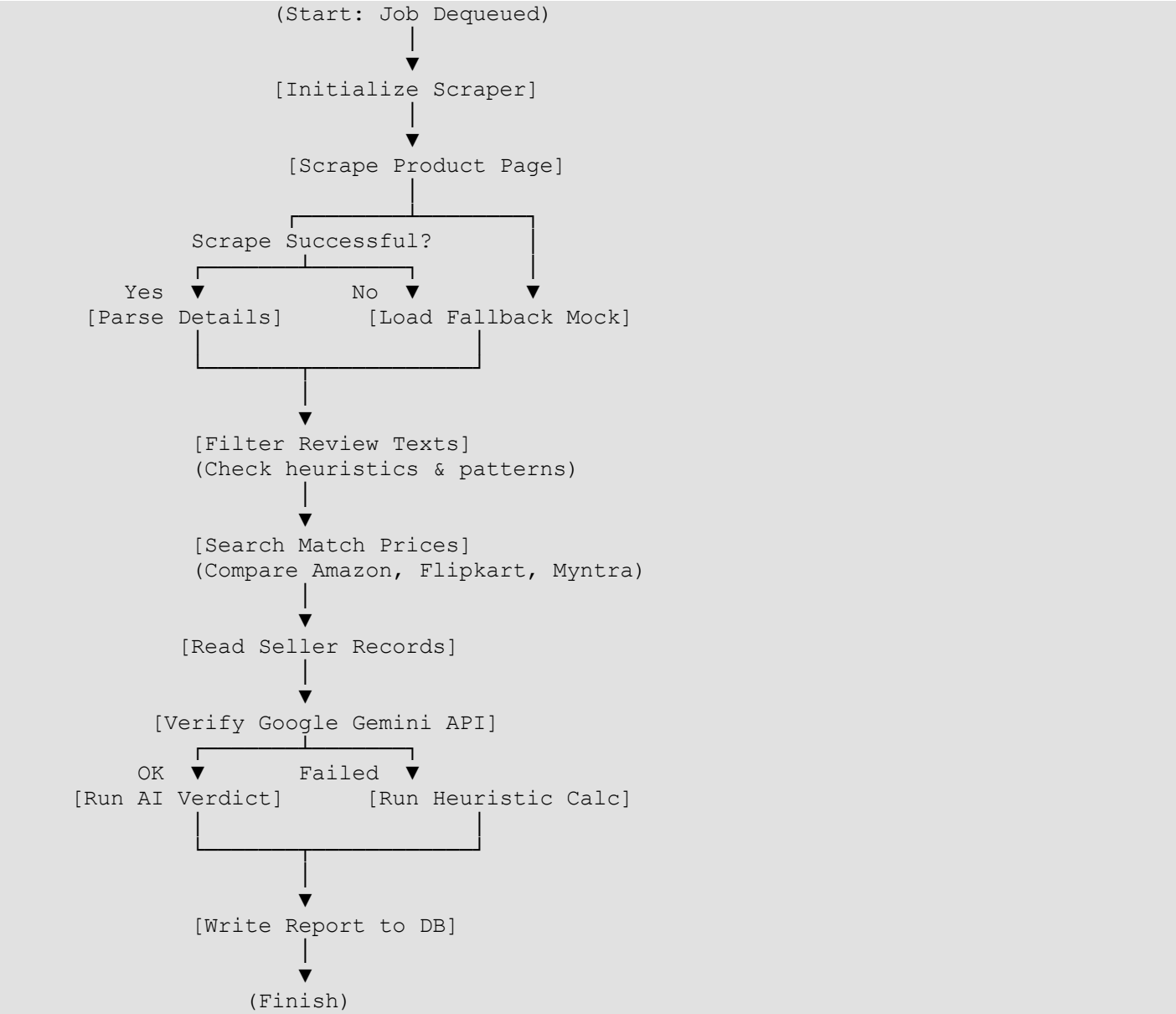

Fig: 5.5. Worker Process Activity Diagram

5.6 OUTPUT

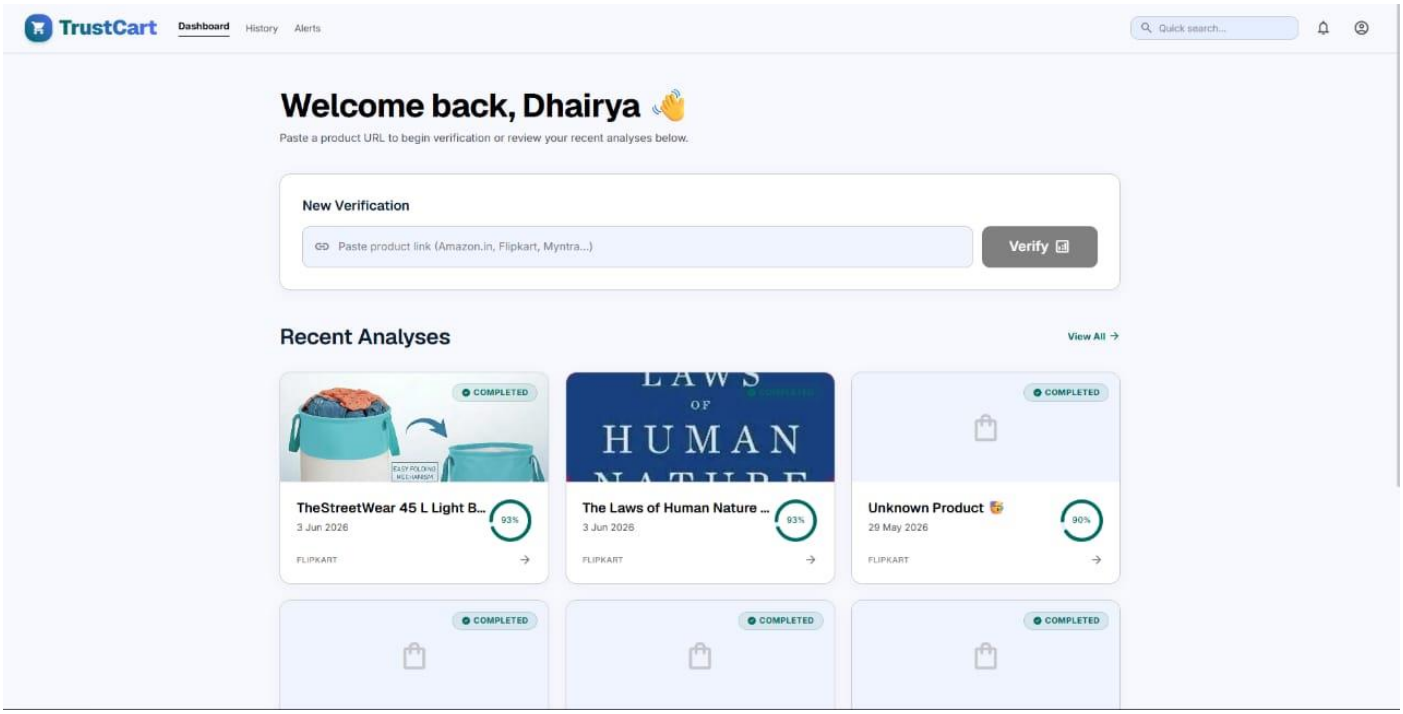

Fig 5.6.1

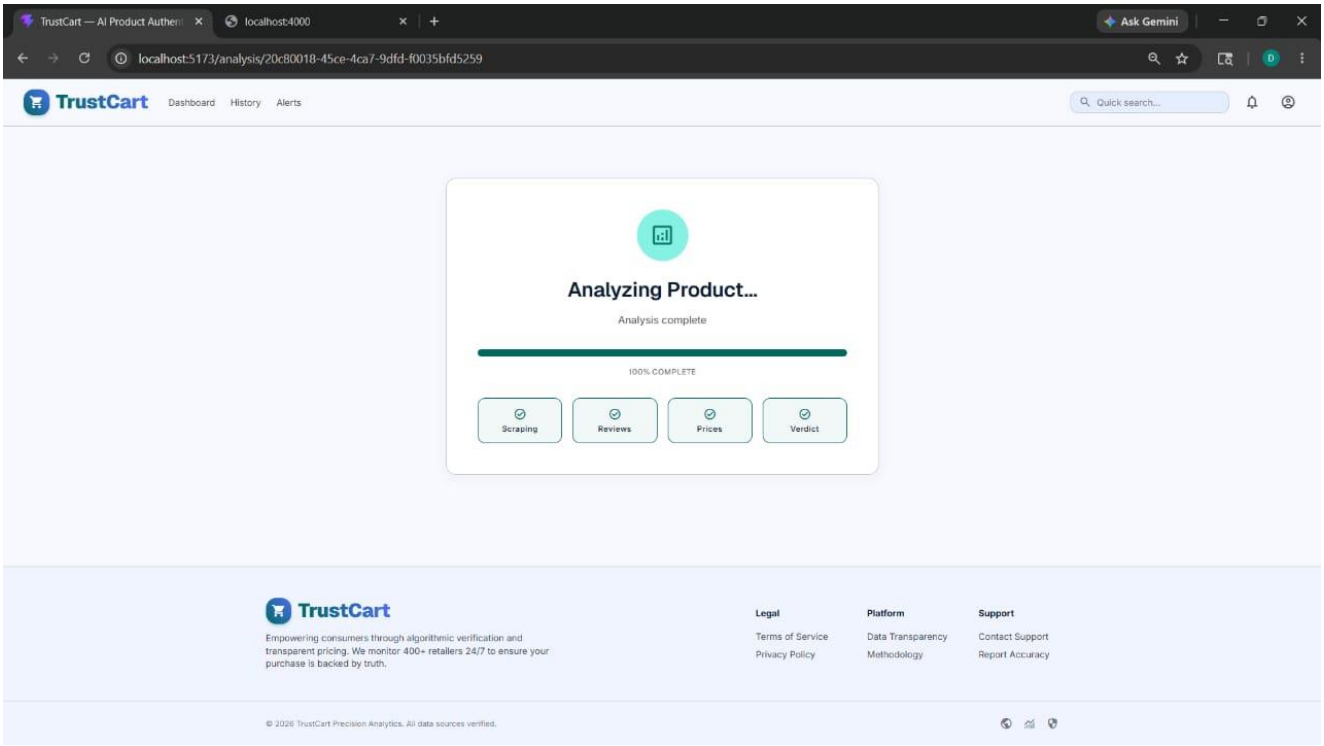

Fig 5.6.2

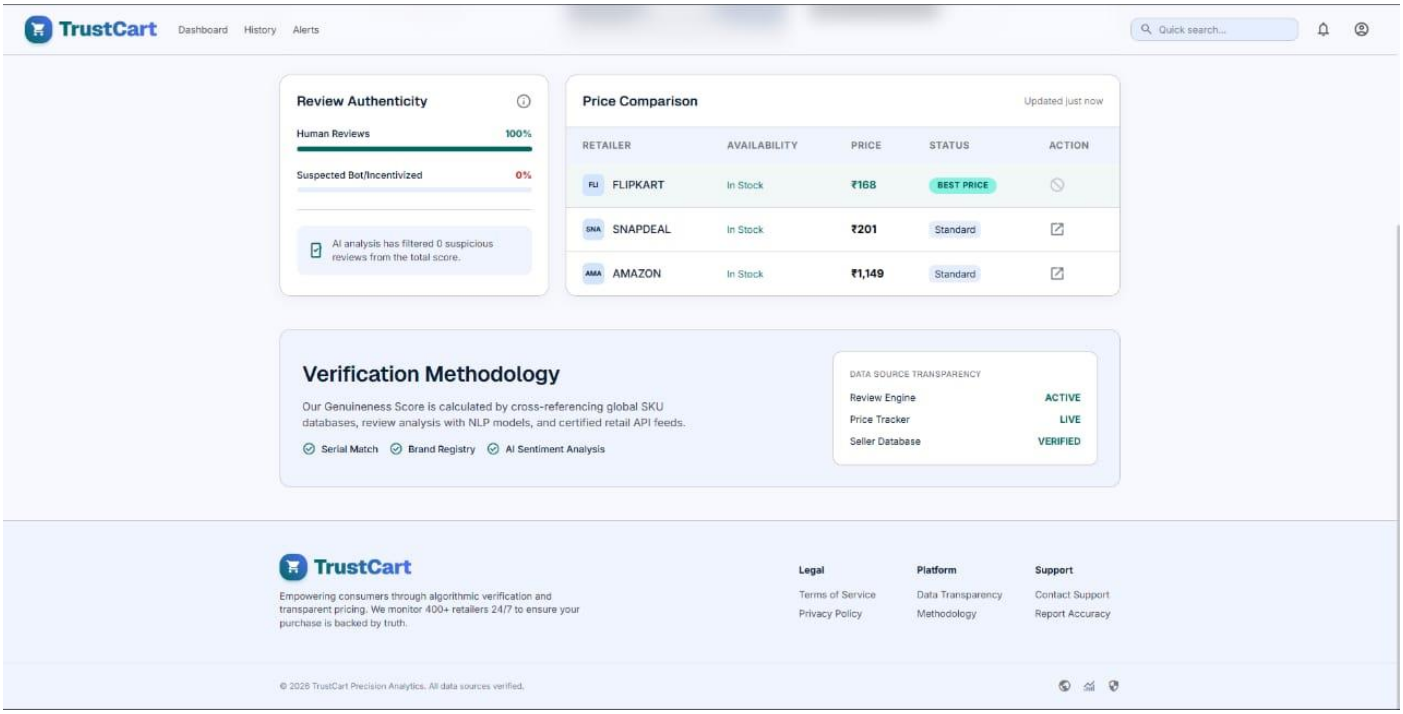

Fig 5.6.3

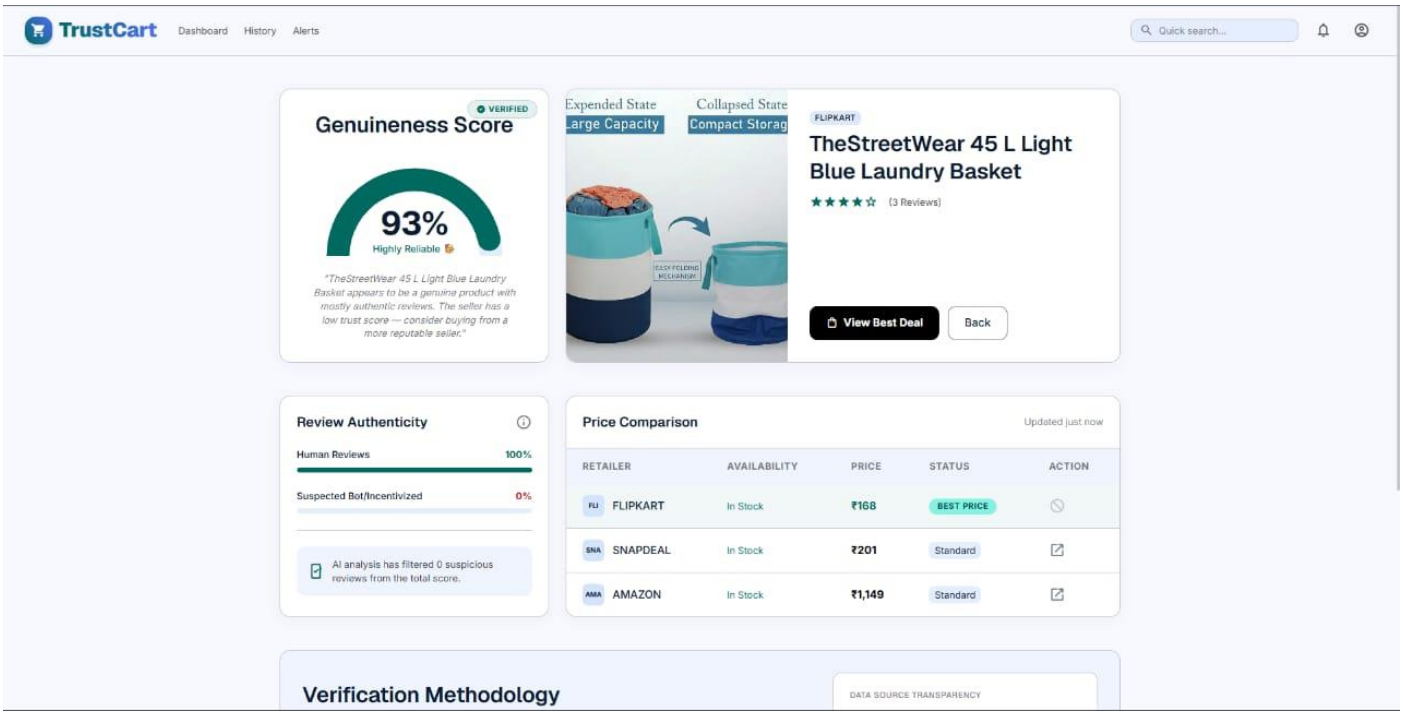

Fig 5.6.4

CHAPTER 6

CONCLUSION, LIMITATIONS AND FUTURE WORK

6.1 SUMMARY OF COMPLETED WORK

During this project, we successfully developed and deployed **TrustCart**, an AI-powered e-commerce auditing platform. The core implementation achievements include:

- **Relational Database Schema**: Implemented a structured database using PostgreSQL and Prisma ORM to record users, analyses, and price alerts.
- **Background Queue Management**: Set up Redis and BullMQ to handle resource-heavy web crawling tasks asynchronously, ensuring the main application remains fast and responsive.
- **Dynamic Web Crawlers**: Built Playwright-based crawlers capable of parsing dynamic page elements from major e-commerce platforms.
- **LLM Synthesis Layer**: Integrated Google Gemini using LangChain to analyze compiled details and output structured buy/avoid recommendations.
- **Premium Dashboard Interface**: Developed a light-themed React dashboard featuring clean typography, semicircular progress gauges, and interactive charts.

6.2 Limitations

- The system currently uses **HTTP polling** for status updates, which introduces slight delays and increases server load.
- Users must **manually enter the product URL**, as browser extension support is not yet available.
- The platform cannot **detect networks of fraudulent sellers** across multiple e-commerce platforms.
- Price comparison is limited to **supported platforms and a single currency**, without global price comparison.

6.3. FUTURE SCOPE & RESEARCH DIRECTIONS

The platform's capabilities can be expanded in future updates through several enhancements:

- **WebSocket Integration**: Replace HTTP polling with WebSockets (using Socket.IO) to push analysis updates from background workers to the client interface instantly, reducing database load.
- **Browser Extension Development**: Build a browser extension that reads the current tab's active URL, queries the TrustCart API, and displays the authenticity score in a side panel directly on retailer sites.
- **Graph-Based Fraud Detection**: Create database schemas to map relationships between third-party sellers using shared details (like registration addresses).

REFERENCES

1.Jindal, N., & Liu, B. (2008). Opinion spam and analysis. *Proceedings of the International Conference on Web Search and Web Data Mining (WSDM)*, 219–230.

2.Mukherjee, A., Liu, B., & Glance, N. (2012). Spotting fake reviewer groups in consumer reviews. *Proceedings of the 21st International Conference on World Wide Web (WWW)*, 191–200.

3.Ott, M., Choi, Y., Cardie, C., & Hancock, J. T. (2011). Finding deceptive opinion spam by any stretch of the imagination. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, 309–319.

4.Li, F., Huang, M., Yang, Y., & Zhu, X. (2014). Learning to identify review spam. *Proceedings of the 22nd International Conference on World Wide Web (WWW)*, 1189–1198.

5.Zhang, Z., & Chen, X. (2019). Detecting fake reviews using deep learning and sentiment analysis. *Journal of Intelligent & Fuzzy Systems*, 37(6), 7251–7263.

6.Baly, R., Karadzhov, G., Alexandrov, A., Glass, J., & Nakov, P. (2018). Predicting factuality of reporting and bias of news media sources. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 352–362.

7.Hugging Face Transformers. (2023). *Hugging Face Documentation*. Retrieved from https://huggingface.co/docs

8.Scikit-learn: Machine Learning in Python. (2023). *Scikit-learn Documentation*. Retrieved from https://scikit-learn.org

9.NLTK Project. (2023). *Natural Language Toolkit Documentation*. Retrieved from https://www.nltk.org

10. Python Software Foundation. (2023). *Python Documentation*. Retrieved from https://www.python.org